$\mathcal{D}^2$ SD: Accelerating Speculative Decoding with Dual Diffusion Draft Models

Liyuan Zhang[1], **Jiarui Zhang**[2,3,5], **Jinwei Yao**[4], **Ran Yan**[3,5], **Yuchen Yang**[2], **Jiahao Zhang**[1], **Tongkai Yang**[5], **Yi Wu**[2], **Binhang Yuan**[3]

[1]Peking University, [2]Tsinghua University, [3]HKUST, [4]UIUC, [5]Ant Group

Abstract

Speculative decoding accelerates autoregressive large language model inference by drafting multiple tokens and verifying them in a single target-model forward pass. Recent diffusion-based drafters generate an entire block of tokens in parallel but usually commit to a single draft sequence per verification: once the first mismatch occurs, all subsequent draft tokens are discarded, resulting in a limited acceptance rate. Naively batching more draft candidate sequences only introduces a marginal improvement, as redundant or poorly placed branches increase the cost of drafting and verification without proportionally increasing the number of accepted tokens. We propose $\mathcal{D}^2$ **SD**, a **d**ual **d**iffusion draft **s**peculative **d**ecoding framework that organizes candidates into a confidence-guided prefix tree, where the first diffusion drafter generates a block along with per-position confidence scores that are used to identify the most likely rejection boundary and select the top-K prefix ranges for recovery; the second variable-prefix diffusion drafter re-anchors at each selected prefix and proposes alternative continuations in one batched pass; the resulting shared-prefix candidates are jointly verified via cascade attention. Empirically, $\mathcal{D}^2$ **SD** shows clear improvements over both the underlying diffusion approach and strong autoregressive speculative decoding baselines.

Code: https://github.com/catnanami/D2-SD

1 Introduction

Autoregressive large language models (LLMs) achieve strong performance [1–3], but the decoding computation remains inherently serial and memory-bandwidth bounded: each new token requires one forward pass through the original model. Speculative decoding alleviates this bottleneck by exploiting idle GPU compute to parallelize generation with a cheaper drafter, to propose multiple tokens in a single verification step over the original model (a.k.a. target model) [4, 5].

One fundamental challenge of speculative decoding lies in the sequential nature of its verification protocol: due to the *longest-correct-prefix rule* (i.e., the speedup is determined by the longest prefix of the draft token sequence(s) accepted via verification), a single mismatch at an early position i immediately invalidates all subsequent draft tokens in the sequence, regardless of their individual correctness. Note that this inherited issue creates a non-uniform importance across drafting positions—errors near the "anchor" (i.e., the last verified token) are catastrophic, while errors near the end of a block could be negligible. Consequently, simply increasing the drafting budget (i.e., generating more tokens) does not guarantee end-to-end speedup unless the additional compute is strategically allocated to minimize early rejections.

To address this, prefix tree-based speculative decoding (e.g., SpecInfer [6], Sequoia [7], and EAGLE-3 [8])

organizes candidate sequences into branching structures to explore multiple continuations. While effective at increasing acceptance length, these methods are fundamentally limited by their autoregressive drafting phase. This sequential generation imposes a heavy "drafting tax" that scales poorly with the candidate budget: as the token budget increases from 4 to 16, the drafting overhead can balloon from 7ms to 24ms [9]. This serial bottleneck creates a diminishing performance boost where the time spent constructing a prefix tree with more candidate sequences often offsets the latency gains from its higher acceptance rate. Consequently, even with state-of-the-art verification kernels like tree-structured Flash Attention [10, 11], the inherent latency of step-by-step drafting prevents these systems from effectively scaling to larger candidate batches in latency-sensitive applications.

Conversely, parallel drafting methods like DFlash [9] leverage a diffusion draft model to generate an entire block of tokens in a single forward pass, drastically reducing drafting latency. However, existing diffusion-style drafters typically apply a uniform computational budget across all positions within a block. This is inherently suboptimal: while early positions are often strongly constrained by the verified prefix, later positions face increasing predictive difficulty and require more diverse exploration. Our empirical study reveals that this computational mismatch leads to a "scaling wall" (Table 1): simply increasing the block size γ yields diminishing or even negative returns once $\gamma \geq 24$, as an early error in a long, linear block renders the remaining extended budget useless. This bottleneck motivates our investigation into *how to accelerate speculative decoding using diffusion-based draft model(s) with sophisticated construction and organization of draft sequences*.

Table 1 DFlash acceptance length (tokens per forward pass, TPF) on Qwen3-8B as the block size γ grows; each row uses a decay-matched optimal training schedule for γ. TPF rises sharply from γ=8 to γ=16, saturates between γ=16 and γ=24, and *decreases* at γ=32 (↓). Simply enlarging the block thus exhibits diminishing and eventually negative returns—the drafted positions must instead be organized into a more effective structure.

Block size γ	MATH-500 TPF	GSM8K TPF
8	5.04	5.00
16	6.05	5.95
24	6.01	6.00
32	5.85↓	5.93↓

Enabling parallel drafting with tree-structured budget allocation is non-trivial. The most intuitive baseline—generating K independent candidates by sampling from the same diffusion drafter forward pass—fails to deliver meaningful gains. These candidates exhibit high error homogeneity: because they share the same underlying probability distribution, they tend to collapse onto the same high-confidence paths and collectively fail at the same difficult decision boundaries. As shown in our ablation (Table 2, full results in Section 4.3), adding K=4 naive branches to DFlash on GSM8K improves the acceptance length by only 0.37 tokens (TPF 6.96→7.33), a marginal gain that is easily offset by the increased verification overhead, whereas $\mathcal{D}^2$ **SD** lifts it by 2.25 tokens (to 9.21) under the same branching budget. This underscores that effective branching requires more than just increasing the sample count; it requires statistically informed recovery at likely rejection points using a model with a distinct inductive bias.

Table 2 TPF on GSM8K (Qwen3-8B, γ=16, K=4): adding K naive resampled branches to DFlash lifts TPF by only 0.37, while $\mathcal{D}^2$ **SD** raises it by 2.25 under the same branching budget; full multi-dataset ablation in Section 4.3.

Method	GSM8K TPF
DFlash	6.96
+ K naive samples	7.33
$\mathcal{D}^2$ **SD**	**9.21**

Towards this end, we propose $\mathcal{D}^2$ **SD**, a <u>d</u>ual <u>d</u>iffusion drafting paradigm to accelerate <u>s</u>peculative <u>d</u>ecoding, where our method differs by explicitly constructing a prefix-tree-like candidate set from the first diffusion drafter's confidence scores, so that re-drafting by the second specially trained drafter is performed only at

likely rejection boundaries rather than through uniform, heuristic, or computationally expensive branching. Concretely, we make the following contributions:

Contribution 1. We design a dual-draft paradigm: First, a DFlash-style diffusion drafter generates an entire block in one parallel pass and produces per-position confidence scores. These confidence scores are then used to estimate the most likely rejection locations and to select the top-K prefix ranges with the most promising recovery. Second, we introduce a variable-prefix diffusion drafter that re-anchors at those selected prefix positions and proposes alternative suffixes in parallel, thereby converting a single diffusion draft into a structured set of shared-prefix candidates.

Contribution 2. We implement this dual-draft paradigm with special training receipts and optimizations: To support this second stage, we design a dedicated training recipe that exposes the model to variable-length visible prefixes and applies an exponentially decaying training loss that emphasizes errors near the anchor. To enable efficient speculative decoding, we verify the resulting branches using cascade attention, allowing the method to reuse shared-prefix computation rather than replicating long KV prefixes across candidates.

Contribution 3. We conduct a concrete empirical study to demonstrate clear improvements over strong speculative decoding baselines. For example, on Qwen3-8B model with a default configuration of (γ=16) and (K=4), $\mathcal{D}^2$ **SD** increases the average greedy-decoding speedup from ($4.16\times$) to ($4.98\times$) relative to standard autoregressive decoding, while raising the average acceptance length from 5.31 to 7.05 when compared with DFlash.

2 Preliminaries

Speculative decoding speedup. Speculative decoding accelerates autoregressive inference of a target model $\mathcal{M}_t$ by introducing a lightweight draft model $\mathcal{M}_d$. In each decoding cycle, $\mathcal{M}_d$ proposes γ candidate tokens, which $\mathcal{M}_t$ verifies in a single parallel forward pass and accepts up to the longest correct prefix. Following [12], the average per-token latency of speculative decoding can be formulated as:

$$L \;=\; \frac{T_{\text{draft}} + T_{\text{verify}}}{\alpha}, \tag{1}$$

where T_{draft} is the time spent producing the draft block, T_{verify} is the cost of one verification pass, and $\alpha \in [1, \gamma{+}1]$ is the expected number of tokens committed per cycle (the mean acceptance length, including the bonus token returned by $\mathcal{M}_t$ at the rejection boundary). Letting L_{target} denote the per-token latency of standard autoregressive decoding with $\mathcal{M}_t$, the resulting wall-clock speedup is

$$\eta \;=\; \frac{L_{\text{target}}}{L} \;=\; \frac{\alpha \cdot L_{\text{target}}}{T_{\text{draft}} + T_{\text{verify}}}. \tag{2}$$

This expression makes the trade-off explicit: speedup grows either by raising the acceptance length α or by reducing the per-cycle drafting/verification overhead $T_{\text{draft}}{+}T_{\text{verify}}$. Block-diffusion drafters such as DFlash [9] drive T_{draft} down by emitting a γ-token block in a single forward pass, but their reliance on a single draft sequence caps α well below $\gamma{+}1$ once an early mismatch occurs. $\mathcal{D}^2$ **SD** targets exactly this gap: it raises α by re-drafting at the predicted rejection boundary while keeping $T_{\text{draft}}{+}T_{\text{verify}}$ nearly unchanged via a single batched second-draft pass and a shared-prefix joint verification.

DFlash. DFlash [9] is a block-diffusion speculative decoding framework that drafts γ tokens in a single parallel forward pass; panel (a) of Figure 1 illustrates one decoding cycle as a drafter $\rightarrow$ target flow. *Drafting*: The drafter consumes a length-γ input whose position 0 holds the anchor—the bonus token returned by the previous verification step—and whose positions $1, \ldots, \gamma{-}1$ are all [MASK]. Conditioned on the target hidden features of the most recently accepted tokens—multi-layer target representations concatenated and projected by a learned FC layer, then injected into the Key and Value projections of every drafter layer—the drafter decodes all $\gamma{-}1$ mask positions jointly in one parallel pass, with mask tokens attending bidirectionally to the target's representations. We highlight that the target model then verifies the drafted block by prefix matching, accepting tokens up to (but not including) the first mismatch and discarding the rest; the same forward pass also returns the next anchor (the target's prediction at the rejection boundary) and refreshed target hidden features that seed the next cycle.

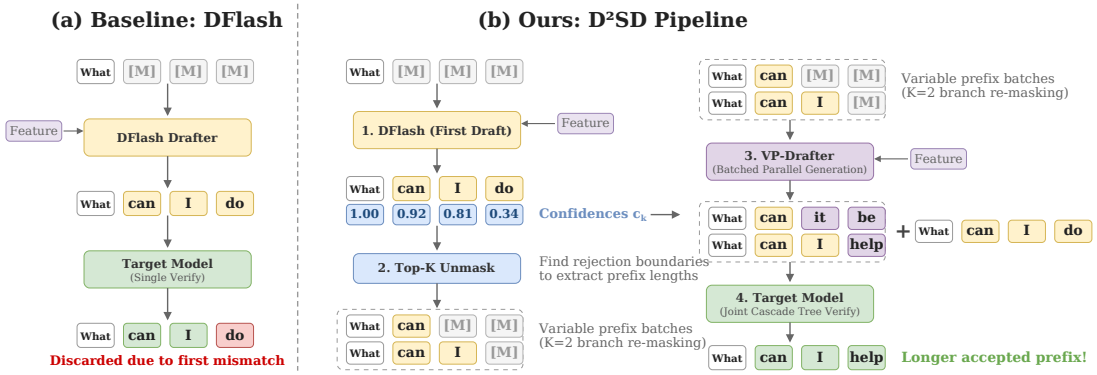

Figure 1 **(a) Baseline DFlash** on a $\gamma = 4$ block. The DFlash drafter consumes the bonus-token anchor at position 0 and $\gamma{-}1$ mask positions, and predicts all mask positions in a single parallel pass conditioned on target hidden features via KV injection. The target model then verifies the drafted block by prefix matching, accepting tokens up to (but not including) the first mismatch and discarding the rest. **(b) Our $\mathcal{D}^2$ SD pipeline** on the same $\gamma{=}4$ block with $K{=}2$ branches. **(1) DFlash (first draft)** runs as in (a), producing a first draft together with per-position confidences c_k. **(2) Top-K Unmask** scores every prefix length i with $r(i)$ (Eq. 4) and selects the K most-likely rejection boundaries, yielding K variable-prefix batches that retain the chosen prefix and re-mask the remainder. **(3) VP-Drafter** fills in all K batches in one batched forward pass, again conditioned on target hidden features. **(4) Target Model** jointly verifies the $K + 1$ candidates (the first-draft block plus K second-draft branches) using shared-prefix cascade attention, and commits the longest accepted prefix across branches (see Section 3.3 for details).

Limitation. DFlash acceptance length is fundamentally bounded by the *longest correct prefix*: once a mismatch occurs at some position, all subsequent draft tokens are discarded regardless of their quality. This observation motivates our dual-draft design, which re-drafts from the rejection boundary to recover these otherwise-lost positions.

3 $\mathcal{D}^2$ SD

We present $\mathcal{D}^2$ **SD** (Dual-Draft Diffusion Speculative Decoding), designed to address the two failure modes identified in the introduction: the scaling wall of single-block diffusion drafting, where enlarging γ yields diminishing or negative returns (Table 1); and the error homogeneity of resampling K branches from the same DFlash forward pass. Our remedy has two components: rather than extend the draft block, we re-anchor it at positions where the first draft is most likely to fail (Sections 3.1–3.3); and rather than reuse DFlash for the second branch, we introduce a structurally distinct VP-Drafter (Variable-Prefix Drafter) trained to produce continuations from any prefix length within the block (Section 3.4), giving the cascade a different inductive bias from the first draft. Section 3.1 explains how we use the first drafter's confidence to estimate where the rejection boundary is likely to occur; Section 3.2 turns this estimate into a top-K set of shared-prefix branches; Section 3.3 describes the end-to-end inference pipeline; and Section 3.4 presents the variable-length prefix training procedure.

3.1 Estimating the Rejection Boundary from Drafter Confidence

The static, uniform allocation from DFlash does not match the verification rule: predictive difficulty is non-uniform across positions within a block. Enlarging γ does not address this mismatch, since each position still receives the same per-position compute—the harder positions that bound acceptance are no better resourced than before, the rejection boundary does not move, and the added tokens are wasted (Table 1). $\mathcal{D}^2$ **SD** replaces this static budget with a dynamic, difficulty-aware one: we re-anchor a second drafter at the predicted rejection boundary, directing an additional forward pass and a distinct inductive prior to the positions where the first draft was most likely to fail. Re-drafting too early wastes this extra capacity on positions that would have already been accepted, while re-drafting too late, on the other hand, leaves the actual rejection inside

the kept prefix. Since the rejection boundary is unknown until verification, the second drafter must predict where the first draft will fail, and the remainder of this section shows that a lightweight drafter-internal signal can do so reliably.

We show that the first drafter's own per-position probability mass is sufficient. At each cycle DFlash produces a categorical $p_k(\cdot)$ at every block position $k = 1, \ldots, \gamma{-}1$ and we greedy-sample $\hat{t}_k = \arg\max_v p_k(v)$; we define the *confidence* of position k as

$$c_k \;=\; p_k(\hat{t}_k) \;=\; \max_v p_k(v). \tag{3}$$

Figure 2a shows that c_k is well-calibrated on GSM8K—the empirical accept rate tracks the diagonal across $[0, 1]$ and is slightly conservative in the high-confidence regime. Although this calibration property has been reported for autoregressive drafters [13, 14], to our knowledge it has not been verified for diffusion-based block drafters; our result shows that a target-conditioned block-diffusion drafter inherits the same property, letting us treat each c_k as a reliable estimate of $\Pr(\hat{t}_k \text{ accepted} \mid \hat{t}_{<k} \text{ accepted})$. Assuming conditional independence given the prefix, the probability that exactly i tokens are accepted ($L = i$) is

$$r(i) \;=\; \underbrace{\prod_{k=1}^{i} c_k}_{\text{first } i \text{ accepted}} \;\cdot\; \underbrace{(1 - c_{i+1})}_{\text{position } i+1 \text{ rejected}}, \quad i = 0, 1, \ldots, \gamma{-}2. \tag{4}$$

In other words, $r(i)$ turns the drafter's internal confidence into a posterior over rejection boundaries, telling us where the recovery budget should be spent.

3.2 Top-K Shared-Prefix Branch Selection

Section 3.1 produces a per-cycle posterior $r(i)$ over rejection boundaries; we now turn it into an actual branch set. As shown in the introduction (Table 2), drawing K extra candidates from the same DFlash forward recovers only a small slice of the headroom, because all draws inherit the same per-position categorical and tend to collapse onto the same high-confidence paths. Effective branching, therefore, must answer two separate questions: where each branch should diverge from the first draft, and which model produces its continuation. We answer the first question here using $r(i)$; the second—a structurally distinct VP-Drafter that gives the cascade a different inductive bias—is deferred to Section 3.4.

Committing to the single $\arg\max_i r(i)$ would forfeit most of the recovery opportunity, because $r(i)$ is typically diffuse—several prefix lengths receive comparable mass. We therefore retain the top-K prefix lengths,

$$\mathcal{S} \;=\; \underset{i \in \{0, \ldots, \gamma-2\}}{\text{Top-K}} \; r(i), \tag{5}$$

and launch one second-draft branch from each. Branch $i \in \mathcal{S}$ retains the anchor and the first i DFlash tokens as its visible prefix and re-drafts the remaining $\gamma{-}1{-}i$ positions with the VP-Drafter (Figure 2b). By construction, every branch shares its first $i{+}1$ tokens with the corresponding DFlash prefix, so the resulting candidate set has exactly the shared-prefix structure that cascade-style joint verification (Section 3.3) can exploit—inheriting the verification-amortization benefit of tree-based speculative decoding [6, 7, 14, 15] without paying their autoregressive drafting tax.

(a) Confidence vs. accept rate. **(b)** Top-K unmask procedure.

Figure 2 **(a)** DFlash drafter confidence (Eq. 3) versus empirical accept rate. Each bar is the mean accept rate of draft tokens whose confidence falls into the corresponding bin; the red dashed line is $y = x$. The drafter is near-diagonally calibrated and slightly conservative in the high-confidence regime. **(b)** Top-K unmask with $K = 4$ on a $\gamma = 6$ block: we score every candidate prefix length with $r(i)$ in Eq. (4) and select the four highest (green checks). Each selected prefix yields one second-draft input that retains the prefix and re-masks the rest; all branches are batched into one VP-Drafter forward pass.

3.3 End-to-End Inference Pipeline

A full $\mathcal{D}^2$ **SD** decoding cycle combines the DFlash first draft, the VP-Drafter second draft, and joint verification. Each cycle consumes the verified KV caches of the target and the two drafters, along with the bonus token from the previous cycle, and emits a new batch of accepted tokens. Panel (b) of Figure 1 illustrates one such cycle on a $\gamma = 4$ block; the four stages are described below.

(i) First draft (DFlash). As in standard DFlash, the drafter builds a length-γ masked block: position 0 holds the bonus token (the anchor) and positions $1, \ldots, \gamma{-}1$ are [MASK]. Conditioned on the target hidden features from the previous cycle, DFlash predicts all $\gamma - 1$ mask positions in one pass, yielding tokens $\hat{t}_1, \ldots, \hat{t}_{\gamma-1}$ with confidences $c_1, \ldots, c_{\gamma-1}$.

(ii) Top-K unmask. From the confidences we compute $r(i)$ (Eq. 4) for each $i \in \{0, \ldots, \gamma{-}2\}$ and pick the top-K prefix lengths $\mathcal{S}$ (Eq. 5); each $i \in \mathcal{S}$ becomes a re-anchoring position.

(iii) Second draft (VP-Drafter, batched). For every $i \in \mathcal{S}$, the second-draft input retains the anchor and the first i DFlash tokens as the visible prefix and re-masks positions $i{+}1, \ldots, \gamma{-}1$. The K branches are stacked into a single batch and processed by the VP-Drafter in one parallel pass, reusing the same target hidden features as DFlash. Each output branch agrees with DFlash on its first $i{+}1$ tokens and proposes alternative continuations for the remainder.

(iv) Joint verification and acceptance. Together with the original DFlash branch, the target model jointly verifies the $K{+}1$ candidates in a single forward pass. We then run a longest-accepted-prefix search across branches and commit the longest accepted prefix; the target's prediction at the rejection position becomes the next anchor (the bonus token).

3.4 Training the VP-Drafter

The VP-Drafter shares its architecture with DFlash—a lightweight Qwen-based block-diffusion model with KV-injected target features—but uses a different masking schedule designed to support <u>any</u> prefix length within the block.

At inference, the unmask procedure of Section 3.2 draws K different prefix lengths from $\{0, \ldots, \gamma{-}2\}$, and the second drafter must produce sensible continuations from each of them. DFlash, by contrast, is trained with a single fixed anchor at position 0; reusing it for the second stage would force extrapolation to unseen prefix lengths, sharply degrading acceptance as the prefix grows.

For each training example we sample a prefix length l from a truncated geometric prior

$$\Pr(l = j) \;\propto\; \beta^j, \quad j = 0, 1, \ldots, \gamma{-}2, \tag{6}$$

with $\beta \in (0,1)$ biasing toward shorter prefixes. The first $l{+}1$ positions are filled with ground-truth tokens and positions $l{+}1, \ldots, \gamma{-}1$ are masked. The drafter then predicts the masked positions in parallel, conditioned on target hidden features extracted from the same teacher pass used by DFlash. Because verification accepts tokens sequentially from the anchor outward, errors on positions closer to the anchor are strictly more costly than errors on later positions. We therefore train with an exponentially decayed cross-entropy that weights each masked position by its distance from the anchor,

$$\mathcal{L}_{\text{VP}} \;=\; -\frac{\sum_{k=l+1}^{\gamma-1} w_k \,\log p_k(t_k^\star)}{\sum_{k=l+1}^{\gamma-1} w_k}, \qquad w_k \;=\; \exp\!\left(-\frac{k-l-1}{\tau}\right), \tag{7}$$

with decay rate τ and the expectation taken over data and the prior in Eq. (6).

4 Experiments

4.1 Setup

Models. We use Qwen3-8B and GPT-OSS-20B as the target model throughout. All experiments are conducted on NVIDIA H200 GPUs unless otherwise noted.

Benchmarks. We evaluate on eight tasks spanning three categories: **Math**—GSM8K [16], MATH [17], and AIME25 [18]; **Code**—HumanEval [19], MBPP [20], and LiveCodeBench [21]; **Chat**—MT-Bench [22] and Alpaca [23].

Training data. All the EAGLE-3, DFlash and the VP-Drafter are trained on the full PerfectBlend[1] dataset [24], which aggregates diverse instruction–response pairs across mathematics, code, chat, and instruction following domains, unless otherwise noted. To ensure the draft models are aligned with the target distribution, all training responses are generated by the target model itself.

Metrics. We measure the *mean acceptance length* α, defined as the average number of tokens accepted per decoding cycle, and the *wall-clock speedup*, defined as the ratio of the autoregressive baseline's per-token decode latency to that of the speculative method.

Implementation. All models are served in BF16 precision with FlashAttention-2 [25]. Cascade attention is implemented with flashinfer [26] kernels and pre-warmed CUDA graphs for both the VP-Drafter and the target verification pass. The default configuration uses second-draft branch sizes $\gamma{=}16$ and $K{=}4$. We use HuggingFace Transformers as the model backbone and train the models on the SpecForge[2].

4.2 End-to-End Results

Tables 3 and 4 report wall-clock speedup and mean acceptance length for DFlash, EAGLE-3 [8], and $\mathcal{D}^2$ **SD** ($K{=}4$, $\gamma{=}16$) on Qwen3-8B and GPT-OSS-20B under greedy ($T{=}0$) and multinomial ($T{=}1$) decoding. Across both targets and both decoding regimes, $\mathcal{D}^2$ **SD** consistently improves over DFlash on both metrics, with the largest gains on math and code where tighter rejection boundaries give the second drafter more headroom to recover, and smaller gains on open-ended chat where the boundary posterior $r(i)$ is more diffuse. Against the autoregressive EAGLE-3, $\mathcal{D}^2$ **SD** wins on wall-clock speedup on every task even when EAGLE-3 occasionally reaches a slightly longer α, confirming that the latency advantage of parallel block drafting outweighs EAGLE-3's marginal acceptance edge. The advantage is amplified on the larger GPT-OSS-20B target, where heavier per-step verification makes each token recovered by re-anchoring translate into a larger wall-clock saving; under $T{=}1$ all methods degrade as draft–target agreement weakens, but $\mathcal{D}^2$ **SD** retains a consistent lead.

[1] https://huggingface.co/datasets/mlabonne/open-perfectblend
[2] https://github.com/sgl-project/SpecForge

Table 3 Wall-clock speedup over autoregressive decoding and mean acceptance length (α) on Qwen3-8B (γ=16, K=4) under greedy (T=0) and sampling (T=1) decoding. EAGLE-3 [8], DFlash, and D^{2}SD are measured on the same public Qwen3-8B checkpoint with matched hardware and decoding hyperparameters; we report end-to-end wall-clock speedup and mean acceptance length from our runs at both temperatures. Row-wise averages are arithmetic means over the eight datasets.

	Greedy (T=0)						**Sampling** (T=1)					
	Speedup			Acc. Len.			Speedup			Acc. Len.		
Dataset	DFlash	EAGLE-3	D^{2}SD	DFlash	EAGLE-3	D^{2}SD	DFlash	EAGLE-3	D^{2}SD	DFlash	EAGLE-3	D^{2}SD
GSM8K	5.51×	4.14×	**6.54**×	6.96	6.36	**9.21**	4.62×	3.72×	**5.32**×	5.84	5.93	**7.38**
MATH	5.11×	4.11×	**6.03**×	6.55	6.44	**8.56**	4.25×	3.81×	**4.90**×	5.38	6.07	**6.81**
AIME25	4.32×	4.03×	**5.10**×	5.49	6.33	**7.26**	3.55×	3.57×	**4.11**×	4.53	5.73	**5.75**
HumanEval	4.39×	3.85×	**5.37**×	5.50	5.92	**7.47**	3.49×	3.47×	**4.05**×	4.35	**5.59**	5.50
MBPP	4.07×	3.92×	**5.03**×	5.09	6.03	**6.96**	3.25×	3.50×	**4.13**×	4.03	**5.57**	4.88
LiveCodeBench	3.88×	3.70×	**4.62**×	4.86	5.93	**6.44**	3.06×	3.10×	**3.53**×	3.85	**5.01**	4.90
MT-Bench	3.23×	3.35×	**3.80**×	4.41	5.22	**5.75**	2.69×	2.79×	**3.09**×	3.52	**4.40**	4.36
Alpaca	2.79×	3.30×	**3.38**×	3.59	**5.09**	4.74	2.50×	2.73×	**2.97**×	3.17	**4.33**	4.02
Average	4.16×	3.80×	**4.98**×	5.31	5.91	**7.05**	3.43×	3.34×	**4.01**×	4.33	5.33	**5.45**

Table 4 GPT-OSS-20B with the public DFlash drafter `z-lab/gpt-oss-20b-DFlash` as draft 1 (https://huggingface.co/z-lab/gpt-oss-20b-DFlash): wall-clock speedup over autoregressive decoding and mean acceptance length (α) (γ=16, K=4) under greedy (T=0) and sampling (T=1). DFlash and D^{2}SD use reruns with this checkpoint (same hardware and decoding hyperparameters otherwise). Row-wise averages are arithmetic means over the eight datasets.

	Greedy (T=0)						**Sampling** (T=1)					
	Speedup			Acc. Len.			Speedup			Acc. Len.		
Dataset	DFlash	EAGLE-3	D^{2}SD	DFlash	EAGLE-3	D^{2}SD	DFlash	EAGLE-3	D^{2}SD	DFlash	EAGLE-3	D^{2}SD
GSM8K	2.83×	2.32×	**6.20**×	3.58	2.81	**8.02**	1.65×	1.17×	**1.94**×	2.00	1.45	**2.49**
MATH	2.48×	2.33×	**5.43**×	3.01	2.84	**7.03**	1.90×	1.18×	**2.27**×	2.29	1.46	**2.91**
AIME25	1.83×	2.26×	**5.18**×	2.14	2.74	**6.51**	1.89×	1.16×	**2.07**×	2.21	1.44	**2.54**
HumanEval	3.50×	2.44×	**6.33**×	4.09	2.99	**7.86**	1.71×	1.17×	**1.93**×	1.97	1.44	**2.37**
MBPP	5.33×	2.44×	**8.87**×	6.21	2.96	**11.94**	1.65×	1.18×	**1.69**×	1.93	1.45	**2.23**
LiveCodeBench	4.72×	2.42×	**6.64**×	5.54	2.93	**9.02**	**1.68**×	1.19×	1.62×	1.97	1.45	**2.17**
MT-Bench	2.65×	2.23×	**2.67**×	3.01	2.70	**3.44**	**1.60**×	1.15×	1.59×	1.80	1.44	**2.04**
Alpaca	4.88×	2.36×	**7.87**×	5.49	2.87	**10.37**	**1.58**×	1.14×	**1.58**×	1.78	1.41	**2.05**
Average	3.53×	2.35×	**6.15**×	4.13	2.85	**8.02**	1.71×	1.17×	**1.84**×	1.99	1.44	**2.35**

4.3 Ablation Study of $\mathcal{D}^2$ SD

To justify the design of $\mathcal{D}^2$ **SD**, we conduct the following ablation studies:

Ablation 1. *Why simply augmenting DFlash with K resampled branches fails?* A natural baseline for the VP-Drafter is to skip the second model and let DFlash itself produce extra candidates: we keep the original argmax draft and append K=4 branches drawn as per-position T=1 multinomial samples from the same drafter forward, verifying all K+1 jointly via cascade attention. This matches $\mathcal{D}^2$ **SD**'s branching budget, so any difference reflects the source of branch diversity rather than verifier capacity. The resampled branches recover only a small slice of the headroom—average α rises from 5.75 to 6.08 and speedup from $4.48\times$ to $5.06\times$, well short of the $\alpha = 7.62$ that $\mathcal{D}^2$ **SD** attains. The reason is structural: all K+1 branches share one drafter forward and therefore the same per-position categorical p_k. At confident positions every sample collapses to the kept argmax, while at the diffuse positions that determine the rejection boundary the K i.i.d. draws cannot reach probability mass outside p_k's support. Resampling the same drafter thus cannot inject information it does not already carry; closing the remaining gap requires a model with a different inductive bias—the VP-Drafter,

trained for variable-length prefixes (Section 3.4).

Table 5 Augmenting argmax DFlash with $K{=}4$ resampled branches yields only modest gains over single-branch DFlash, while $\mathcal{D}^2$ **SD** improves both metrics substantially under the same branching budget. Greedy target decoding ($T{=}0$); each cell is speedup $\times$ / mean acceptance length α.

Dataset	DFlash	$+K$ samples	$\mathcal{D}^2$ **SD**
GSM8K	5.51×/6.96	6.18×/7.33	**6.54×/9.21**
MATH	5.11×/6.55	5.67×/6.82	**6.03×/8.56**
MBPP	4.07×/5.09	4.66×/5.47	**5.03×/6.96**
MT-Bench	3.23×/4.41	3.74×/4.70	**3.80×/5.75**
Avg.	4.48×/5.75	5.06×/6.08	**5.35×/7.62**

Ablation 2. *Why not directly reusing DFlash as the second drafter?* The complementary baseline keeps $\mathcal{D}^2$ **SD**'s full pipeline—top-K confidence unmask, variable-prefix re-anchoring, and joint cascade verification—but reuses the original DFlash model as the second drafter, so the only change from $\mathcal{D}^2$ **SD** is the inductive bias of the second draft. Because DFlash is trained with a fixed anchor at position 0, applying it to the variable-length prefixes the cascade selects forces extrapolation to settings it has never seen.

The cascade structure clearly helps: average α rises to 6.53 ($+14\%$ over single-branch DFlash, against only $+6\%$ for the naive resampling baseline), confirming that re-anchoring at the predicted rejection boundary places branches more effectively than uniform sampling. Wall-clock speedup, however, rises only modestly to $4.69\times$ ($+5\%$), since the second drafter is now a full forward pass rather than a logits view of the first. Full $\mathcal{D}^2$ **SD** still leads by 1.09 tokens of α ($+17\%$) and $0.66\times$ of speedup ($+14\%$)—precisely the gap the variable-prefix training of Eq. (7) buys by calibrating the second drafter to exactly the longer prefixes the cascade most relies on.

Table 6 Reusing DFlash as the second drafter inside $\mathcal{D}^2$ **SD**'s cascade pipeline ($K{=}4$). Confidence-guided branch placement raises α over single-branch DFlash, but the lack of variable-prefix training keeps both metrics below full $\mathcal{D}^2$ **SD**. Each cell is speedup $\times$ / mean acceptance length α.

Dataset	DFlash	DFlash→DFlash	$\mathcal{D}^2$ **SD**
GSM8K	5.51×/6.96	5.76×/7.95	**6.54×/9.21**
MATH	5.11×/6.55	5.28×/7.35	**6.03×/8.56**
MBPP	4.07×/5.09	4.36×/5.90	**5.03×/6.96**
MT-Bench	3.23×/4.41	3.35×/4.91	**3.80×/5.75**
Avg.	4.48×/5.75	4.69×/6.53	**5.35×/7.62**

Ablation 3. *Why not using more cascaded levels?* A natural question is whether cascading the same idea once more buys further headroom. We test this by stacking a third VP-Drafter level on top of $\mathcal{D}^2$ **SD**: for every $i \in \mathcal{S}$, we re-apply the rejection-boundary estimator (Eq. 4) to that branch's second-draft tail, take the single top-1 location j_i, and dispatch a third VP-Drafter forward anchored there. This yields up to $2K{+}1$ shared-prefix candidates that the target jointly verifies in a single pass; reusing the VP-Drafter is natural since its variable-prefix training (Section 3.4) already covers any prefix length in $\{1, \ldots, \gamma{-}2\}$.

The third level buys acceptance length but loses wall-clock speedup (Table 7): averaged across the four datasets, α rises from 7.62 to 8.24 ($+8\%$) while speedup regresses from $5.35\times$ to $5.13\times$ (-4%). The asymmetry is structural: the third pass operates on the residual probability mass the second did not already cover, so its marginal acceptance contribution (0.62 tokens) is only about a third of the second pass's gain over single-branch DFlash (1.87 tokens). Meanwhile both halves of the per-cycle cost grow—an extra VP-Drafter forward in T_{draft} and roughly double the cascade-attention work in T_{verify}—inflating $T_{\text{draft}}{+}T_{\text{verify}}$ by $\sim 13\%$ against only $\sim 8\%$ acceptance gain. Stacking deeper would only sharpen this asymmetry, so $\mathcal{D}^2$ **SD** commits to a single re-anchoring stage.

Table 7 Stacking a third VP-Drafter level on top of $\mathcal{D}^2$ **SD** ($K{=}4$). Every second-draft branch is re-anchored once at its own top-1 predicted rejection boundary, producing up to $2K{+}1 = 9$ shared-prefix candidates per cycle. Mean acceptance length improves modestly, but the extra drafting and verification cost outweighs the marginal recovery, regressing wall-clock speedup. Each cell is speedup $\times$ / mean acceptance length α.

Dataset	$\mathcal{D}^2$ **SD**	+ 3rd draft
GSM8K	**6.54**×/9.21	6.25×/**9.89**
MATH	**6.03**×/8.56	5.79×/**9.27**
MBPP	**5.03**×/6.96	4.85×/**7.58**
MT-Bench	**3.80**×/5.75	3.63×/**6.20**
Avg.	**5.35**×/7.62	5.13×/**8.24**

5 Related Work

Speculative Decoding. Speculative decoding [4, 5] accelerates autoregressive LLM inference by employing a lightweight drafter to propose candidate tokens that are then verified once by the target model, yielding lossless acceleration through rejection sampling. The core question is how to obtain a drafter that is both cheap to run and well aligned with the target distribution. Medusa [15] removes the external drafter altogether by attaching multiple prediction heads to the target LLM. The EAGLE series [8, 14, 27] instead reuses the target's frozen hidden states as drafter input and progressively refines draft quality, currently defining the autoregressive state of the art. Complementary directions include lookahead decoding via Jacobi trajectories [28] and online drafter adaptation to shifting input distributions [29]. An orthogonal *system optimization* amortizes verification cost by jointly checking multiple candidate continuations in a single target pass. SpecInfer [6] introduces tree-structured speculation with shared-prefix tree attention; Sequoia [7] casts tree topology as a constrained optimization that maximizes expected acceptance under a fixed verification budget; EAGLE-2 [14] dynamically constructs context-dependent draft trees; and Medusa-2 [15] verifies candidates sampled from its multiple heads via tree attention.

Diffusion Language Models. Diffusion language models (dLLMs) cast text generation as iterative denoising over a masked sequence, allowing many positions to be decoded in parallel rather than strictly autoregressive. Recent efforts have substantially narrowed the quality gap with autoregressive models: LLaDA-1.5 [30] introduces variance-reduced preference optimization tailored to discrete diffusion, and LLaDA-2.1 [31] further accelerates sampling through token editing. Despite these advances, dLLMs still lag autoregressive counterparts in quality, limiting their adoption as standalone generators. This residual gap, however, is precisely what motivates their use as drafters within speculative decoding: their parallel decoding can be paired with an autoregressive verifier that absorbs any residual quality loss while preserving quality.

Speculative Decoding with Diffusion Draft Models. A growing line of work replaces the sequential drafter with a diffusion-based alternative to overcome the linear drafting cost of autoregressive speculation. DiffuSpec [32] adopts a large pre-trained dLLM as the drafter, achieving long acceptance lengths but incurring memory and latency overhead that partially offsets the speedup. PARD [33] mimics diffusion-style parallel generation with a small, low-cost autoregressive model, yet its limited capacity caps the attainable acceptance length. DFlash [9] introduces a lightweight block-diffusion drafter conditioned on target hidden features via KV injection and reports over $6\times$ lossless speedup; we adopt it as our first-stage drafter. Despite their parallelism, all of the above commit to a single draft sequence per verification step, so an early mismatch discards every downstream token, and the methods do not naturally interoperate with tree-based verification. A very recent concurrent work of [34] extends block-diffusion drafting to draft trees, but generates a large number of branches via uniform resampling, which dilutes the advantages of parallel verification on positions where the first draft was already correct.

6 Conclusion

We presented $\mathcal{D}^2$ **SD**, a dual-diffusion-draft speculative decoding framework that converts a single block-diffusion draft into a structured set of shared-prefix candidates rather than committing to one sequence per verification. A first diffusion drafter emits a γ-token block together with per-position confidences; these confidences localize the most likely rejection boundary, from which the top-K prefix lengths are selected; a variable-prefix diffusion drafter, trained with an anchor-weighted loss to support arbitrary prefix lengths, re-anchors at each selected prefix and proposes alternative continuations in one batched pass; the resulting $K + 1$ shared-prefix candidates are jointly verified by the target model. By construction, this targets the two failure modes that limit existing diffusion drafters—the scaling wall of enlarging γ and the error homogeneity of resampling the same forward—while sidestepping the autoregressive drafting tax that bounds tree-based speculative decoders. The experiment results indicate that confidence-guided shared-prefix recovery—rather than block enlargement or uniform branch resampling—is the right primitive for combining parallel block drafting with the longest-correct-prefix verification rule.

References

[1] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.

[2] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.

[3] Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. arXiv preprint arXiv:2602.15763, 2026.

[4] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In International Conference on Machine Learning, pages 19274–19286. PMLR, 2023.

[5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. arXiv preprint arXiv:2302.01318, 2023.

[6] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, pages 932–949, 2024.

[7] Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable, robust, and hardware-aware speculative decoding. arXiv preprint arXiv:2402.12374, 2024.

[8] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. In The Thirty-ninth Annual Conference on Neural Information Processing Systems, 2025.

[9] Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. arXiv preprint arXiv:2602.06036, 2026.

[10] Zaifeng Pan, Yitong Ding, Yue Guan, Zheng Wang, Zhongkai Yu, Xulong Tang, Yida Wang, and Yufei Ding. Fasttree: Optimizing attention kernel and runtime for tree-structured llm inference. Proceedings of Machine Learning and Systems, 7, 2025.

[11] Jinwei Yao, Kaiqi Chen, Kexun Zhang, Jiaxuan You, Binhang Yuan, Zeke Wang, and Tao Lin. Deft: Decoding with flash tree-attention for efficient tree-structured llm inference. In 13th International Conference on Learning Representations, ICLR 2025, pages 3587–3618. International Conference on Learning Representations, ICLR, 2025.

[12] Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, Vashisth Tiwari, Ruihang Lai, Jinyuan Shi, Ian En-Hsu Yen, Avner May, Tianqi Chen, and Beidi Chen. Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. In International Conference on Learning Representations (ICLR), 2025.

[13] Cunxiao Du, Jing Jiang, Xu Yuanchen, Jiawei Wu, Sicheng Yu, Yongqi Li, Shenggui Li, Kai Xu, Liqiang Nie, Zhaopeng Tu, et al. Glide with a cape: A low-hassle method to accelerate speculative decoding. In International Conference on Machine Learning, pages 11704–11720. PMLR, 2024.

[14] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees. In Proceedings of the 2024 conference on empirical methods in natural language processing, pages 7421–7432, 2024.

[15] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In International Conference on Machine Learning, pages 5209–5235. PMLR, 2024.

[16] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[17] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In J. Vanschoren and S. Yeung, editors, Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks, volume 1, 2021.

[18] Mathematical Association of America. American Invitational Mathematics Examination – AIME 2025. https://maa.org/maa-invitational-competitions, February 2025. Accessed: 2026-05-06.

[19] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.

[20] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.

[21] Naman Jain, Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. 2025:58791–58831, 2025.

[22] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. Advances in neural information processing systems, 36:46595–46623, 2023.

[23] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

[24] Tengyu Xu, Eryk Helenowski, Karthik Abinav Sankararaman, Di Jin, Kaiyan Peng, Eric Han, Shaoliang Nie, Chen Zhu, Hejia Zhang, Wenxuan Zhou, et al. The perfect blend: Redefining rlhf with mixture of judges, 2024.

[25] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In 12th International Conference on Learning Representations, ICLR 2024, 2024.

[26] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. Flashinfer: Efficient and customizable attention engine for llm inference serving. Proceedings of Machine Learning and Systems, 7, 2025.

[27] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: speculative sampling requires rethinking feature uncertainty. In Proceedings of the 41st International Conference on Machine Learning, pages 28935–28948, 2024.

[28] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of llm inference using lookahead decoding. In International Conference on Machine Learning, pages 14060–14079. PMLR, 2024.

[29] Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Online speculative decoding. arXiv preprint arXiv:2310.07177, 2023.

[30] Fengqi Zhu, Rongzhen Wang, Shen Nie, Xiaolu Zhang, Chunwei Wu, Jun Hu, Jun Zhou, Jianfei Chen, Yankai Lin, Ji-Rong Wen, et al. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. arXiv preprint arXiv:2505.19223, 2025.

[31] Tiwei Bie, Maosong Cao, Xiang Cao, Bingsen Chen, Fuyuan Chen, Kun Chen, Lun Du, Daozhuo Feng, Haibo Feng, Mingliang Gong, et al. Llada2. 1: Speeding up text diffusion via token editing. arXiv preprint arXiv:2602.08676, 2026.

[32] Guanghao Li, Zhihui Fu, Min Fang, Qibin Zhao, Ming Tang, Chun Yuan, and Jun Wang. Diffuspec: Unlocking diffusion language models for speculative decoding. arXiv preprint arXiv:2510.02358, 2025.

[33] Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. Pard: Accelerating llm inference with low-cost parallel draft model adaptation, 2025.

[34] Liran Ringel and Yaniv Romano. Accelerating speculative decoding with block diffusion draft trees. arXiv preprint arXiv:2604.12989, 2026.